Student name: Min Xue
Student number: 874430

Task a) Implement the given mathematical programming model in Gurobi for Python

The model are implemented in gurobi with following codes:

```
// Model Construction:
m = gb.Model('Model1')
// Variables implement:
x = m.addVars(range(1,5), lb = 0, vtype = GRB.INTEGER, name = 'x' )
s = m.addVars(range(1,5), lb = 0, vtype = GRB.INTEGER, name = 's' )
// Function implement:
m.setObjective(2*x[1] + 5*x[2] + 3*x[3] + 3*x[4] + 0.25 * (s[1] + s[2] + s[3] + s[4]), GRB.MINIMIZE)
// Constraints implement:
m.addConstrs(x[1] <=600) #constraints 1
m.addConstrs(x[2] <=300) #constraints 2
m.addConstrs(x[3] <=500) #constraints 3
m.addConstrs(x[4] <=400) #constraints 4
m.addConstrs(x[1] - s[1] == 400) #constraints 5
m.addConstrs(x[2] + s[1] - s[2] == 500) #constraints 6
m.addConstrs(x[3] + s[2] - s[3] == 400) #constraints 7
m.addConstrs(x[4] + s[3] - s[4] == 400) #constraints 8
m.addConstrs(x[i] >=0 for i in range(1,5)) #constraints 9
m.addConstrs(s[i] >=0 for i in range(1,5)) #constraints 10
```

Task b) Solve the model and present a decision recommendation with justifications

Model results

After solving the model with ".optimize()", the results are got by ".Attr('x')" in Gurobi, these results for variables are provided in the table below.

Table 1.1. Results for deterministic problem

x_1	x_2	x_3	x_4	s_1	s_2	s_3	s_4
600.0	300.0	400.0	400.0	200.0	0.0	0.0	0.0

Decision recommend

The solution finds that there is only one feasible solution for the problem and the optimized values for each variable are $x_1 = 600$, $x_2 = 300$, $x_3 = 400$, $x_4 = 400$, $s_1 = 200$, $s_2 = 0$, $s_3 = 0$ and $s_4 = 0$.

The decision maker is recommended to have **600 units production with 200 units estimated inventory in the first quarter**, **300 units production with 0 units estimated inventory in the second quarter**, **400 units production with 0 units estimated inventory in the third quarter** and **400 units production with 0 units estimated inventory in the fourth quarter**.

Justification

The reason to recommend decision makers with above recommendations is rooting on the principle with minimum costs. The minimization is reached by reducing the inventory costs, especially those for quarter 2 to quarter 4. The total costs are minimized to 5150 with the above recommendation.

Task c) Implement the given mathematical programming model in Gurobi for Python

The model are implemented in gurobi with following codes:

```
// Model Construction:
so = gb.Model('Model2')
// Variables implement:
za = so.addVars(range(1,4), lb = 0, vtype = GRB.BINARY, name = 'za' )
zb = so.addVars(range(1,4), lb = 0, vtype = GRB.BINARY, name = 'zb' )
r = so.addVars(range(1,8), lb = 0, name = 'r' )
// Function implement:
so.setObjective((0.5*0.3)*r[4] + (0.5*0.7)*r[5] + (0.5*0.4)*r[6] + (0.5*0.6)*r[7], GRB.MAXIMIZE)
// Constraints implement:
so.addConstr(r[1] == 9 - 1 * za[1] - 2 * zb[1]) #constraints 1
so.addConstr(r[2] == 1.08 * r[1] - 3 * za[2] - 2 * zb[2]) #constraints 2
so.addConstr(r[3] == 1.08 * r[1] - 3 * za[3] - 2 * zb[3]) #constraints 3
so.addConstr(r[4] == 1.08 * r[2] + 20 * za[2] + 2.5 * zb[2]) #constraints 4
so.addConstr(r[5] == 1.08 * r[2] + 10 * za[2] + 1 * zb[2]) #constraints 5
so.addConstr(r[6] == 1.08 * r[3] + 5 * za[3] + 25 * zb[3]) #constraints 6
so.addConstr(r[7] == 1.08  *r[3] + 0 * za[3] + 10 * zb[3]) #constraints 7
so.addConstr(za[1] >= za[2]) #constraints 8
so.addConstr(za[1] >= za[3]) #constraints 9
so.addConstr(zb[1] >= zb[2]) #constraints 10
so.addConstr(zb[1] >= zb[3]) #constraints 11
```

Task d) Solve the model and present a decision recommendation with justifications

The results for this stochastic optimization problem are shown in the following chart. In this table, the results for Rs are rounded to 4 digits. The originally printed results are shown in the "Appendix".

Table 1.2. Results for stochastic problem

z_0^A	z_1^A	z_2^A	z_0^B	z_1^B	z_2^B	R_0	R_1	R_2	R_{11}	R_{12}	R_{21}	R_{22}
1.0	1.0	0.0	1.0	1.0	1.0	6.0000	3.4800	4.4800	23.7584	13.7584	29.8384	14.8384

Decision recommendation

The solution finds that there is only one feasible solution for the problem and the optimized values for each variable are $z_0^A = 1.0$, $z_1^A = 1.0$, $z_2^A = 0.0$, $z_0^B = 1.0$, $z_1^B = 1.0$, $z_2^B = 1.0$, $R_0 = 6.0$ and $R_1 = 3.480000000000000004$, $R_2 = 4.48$, $R_{11} = 23.7584$, $R_{12} = 13.758400000000002$, $R_{21} = 29.8384$, $R_{22} = 14.8384$.

Thus, the decision maker can have both project A and B in the scenario 0 and scenario 1, but at scenario 2, the decision maker is recommended to only continue project B. In this case, the cash flow after the first decision for each project is 6 million dollars, the total cash flow in scenario k in period 1 immediately after making the second decision are approximately 3.48 million dollars and 4.48 million dollars respectively for project A and project B. The outcome in scenario ij in period 2 are approximately $R_{11} = 23.7584$, $R_{12} = 13.7584$, $R_{21} = 29.8384$, $R_{22} = 14.8384$ for project A and B respectively.

Justification

The reason to recommend decision makers with above recommendations is based on the principle with maximum outcomes. The maximization was reached by not continue project A in scenario 2. The cash flow can be maximized to 18.7984 with the above recommendation.

Task e) Briefly discuss the role of uncertainty and its effects in this problem

In stochastic optimization problem, random chances implies that the optimization are processed under a set of possible scenarios with uncertainties. For example, in this problem, how the cash flow changes is not 100% known, it could possibly change to any possible scenario with some probabilities and estimates. In scenarios involving problems with few decision variables and few states, it is possible to explicitly list all the possibilities using decision trees. The decision trees are very intuitive, it helps to transfer uncertainties into "certainty", that is the expected value, to some degree. The combination with different decision trees in objective functions with binary variables for optization can figure out the most fitted sub-models for different needs.

The model for this problem assumes that the decision makers are risk-neutral and they only pay attention to the expected value. The project A and B are combined into one case in the optimization function with binary variables to figure out the project combination for each scenario.

Appendix

Problem 1: deterministic optimisation

Implement the model

In [1]:
```
# import neccessary packages
import gurobipy as gb
from gurobipy import GRB
```

In [2]:
```
# create an empty model
m = gb.Model('Model1')
```

```
Set parameter Username
Academic license - for non-commercial use only - expires 2023-02-28
```

In [3]:
```
# add variables: x, s
x = m.addVars(range(1,5), lb = 0, vtype = GRB.INTEGER, name = 'x' )
s = m.addVars(range(1,5), lb = 0, vtype = GRB.INTEGER, name = 's' )
```

In [4]:
```
# set objective function
m.setObjective(2*x[1] + 5*x[2] + 3*x[3] + 3*x[4] + 0.25 * (s[1] + s[2] + s[3] + s[4]), GRB.MINIMIZE)
```

In [5]:
```
# add constraints
m.addConstr(x[1] <=600) #constraints 1
m.addConstr(x[2] <=300) #constraints 2
m.addConstr(x[3] <=500) #constraints 3
m.addConstr(x[4] <=400) #constraints 4
m.addConstr(x[1] - s[1] == 400) #constraints 5
m.addConstr(x[2] + s[1] - s[2] == 500) #constraints 6
m.addConstr(x[3] + s[2] - s[3] == 400) #constraints 7
m.addConstr(x[4] + s[3] - s[4] == 400) #constraints 8
m.addConstrs(x[i] >=0 for i in range(1,5)) #constraints 9
m.addConstrs(s[i] >=0 for i in range(1,5)) #constraints 10
```

Out[5]:
```
(1: <gurobi.Constr *Awaiting Model Update*>,
 2: <gurobi.Constr *Awaiting Model Update*>,
 3: <gurobi.Constr *Awaiting Model Update*>,
 4: <gurobi.Constr *Awaiting Model Update*>)
```

Solve the model

In [6]:
```
# optimization
m.optimize()
```

```
Gurobi Optimizer version 9.5.1 build v9.5.1rc2 (mac64[x86])
Thread count: 2 physical cores, 4 logical processors, using up to 4 threads
Optimize a model with 16 rows, 8 columns and 23 nonzeros
Model fingerprint: 0x81cd48cf
Variable types: 8 continuous, 8 integer (0 binary)
Coefficient statistics:
  Matrix range     [1e+00, 1e+00]
  Objective range  [2e-01, 5e+00]
  Bounds range     [0e+00, 0e+00]
  RHS range        [2e+02, 6e+02]
Presolve removed 16 rows and 8 columns
Presolve time: 0.01s
Presolve: All rows and columns removed

Explored 0 nodes (0 simplex iterations) in 0.02 seconds (0.00 work units)
Thread count was 1 (of 4 available processors)

Solution count 1: 5150

Optimal solution found (tolerance 1.00e-04)
Best objective 5.150000000000e+03, best bound 5.150000000000e+03, gap 0.0000%
```

In [7]:
```
# get solutions
solution_x = m.getAttr('x')
```

In [8]:
```
# print solutions
print(solution_x)
```

```
[600.0, 300.0, 400.0, 400.0, 200.0, 0.0, -0.0, -0.0]
```

Problem 2: stochastic optimisation

Implement the model

In [9]:
```
# create an empty model for this problem
so = gb.Model('Model2')
```

In [10]:
```
# add variables: za,zb and r
za = so.addVars(range(1,4), lb = 0, vtype = GRB.BINARY, name = 'za' )
zb = so.addVars(range(1,4), lb = 0, vtype = GRB.BINARY, name = 'zb' )
r = so.addVars(range(1,8), lb = 0, name = 'r' )
```

In [11]:
```
# set objective function
so.setObjective((0.5*0.3)*r[4] + (0.5*0.7)*r[5] + (0.5*0.4)*r[6] + (0.5*0.6)*r[7], GRB.MAXIMIZE)
```

In [12]:
```
# add constraints
so.addConstr(r[1] == 9 - 1 * za[1] - 2 * zb[1]) #constraints 1
so.addConstr(r[2] == 1.08 * r[1] - 3 * za[2] - 2 * zb[2]) #constraints 2
so.addConstr(r[3] == 1.08 * r[1] - 3 * za[3] - 2 * zb[3]) #constraints 3
so.addConstr(r[4] == 1.08 * r[2] + 20 * za[2] + 2.5 * zb[2]) #constraints 4
so.addConstr(r[5] == 1.08 * r[2] + 10 * za[2] + 1 * zb[2]) #constraints 5
so.addConstr(r[6] == 1.08 * r[3] + 5 * za[3] + 25 * zb[3]) #constraints 6
so.addConstr(r[7] == 1.08  *r[3] + 0 * za[3] + 10 * zb[3]) #constraints 7
so.addConstr(za[1] >= za[2]) #constraints 8
so.addConstr(za[1] >= za[3]) #constraints 9
so.addConstr(zb[1] >= zb[2]) #constraints 10
so.addConstr(zb[1] >= zb[3]) #constraints 11
```

Out[12]:
```
<gurobi.Constr *Awaiting Model Update*>
```

Solve the model

In [13]:
```
# solve the model
so.optimize()
```

```
Gurobi Optimizer version 9.5.1 build v9.5.1rc2 (mac64[x86])
Thread count: 2 physical cores, 4 logical processors, using up to 4 threads
Optimize a model with 11 rows, 13 columns and 34 nonzeros
Model fingerprint: 0xa0f4b244
Variable types: 7 continuous, 6 integer (6 binary)
Coefficient statistics:
  Matrix range     [1e+00, 2e+01]
  Objective range  [1e-01, 3e-01]
  Bounds range     [1e+00, 1e+00]
  RHS range        [9e+00, 9e+00]
Presolve removed 11 rows and 13 columns
Presolve time: 0.00s
Presolve: All rows and columns removed

Explored 0 nodes (0 simplex iterations) in 0.02 seconds (0.00 work units)
Thread count was 1 (of 4 available processors)

Solution count 1: 18.7984

Optimal solution found (tolerance 1.00e-04)
Best objective 1.879840000000e+01, best bound 1.879840000000e+01, gap 0.0000%
```

In [14]:
```
# get solutions
solution_z = so.getAttr('x')
```

In [15]:
```
# print solutions
print(solution_z)
```

```
[1.0, 1.0, 0.0, 1.0, 0.0, 1.0, 6.0, 3.480000000000000004, 4.48, 23.7584, 13.758400000000002, 29.8384, 14.8384]
```